

Problem A. A

Time Limit 1000 ms

Mem Limit 524288 kB

You have a deck of n cards, and you'd like to reorder it to a new one.

Each card has a value between 1 and n equal to p_i . All p_i are pairwise distinct. Cards in a deck are numbered from bottom to top, i. e. p_1 stands for the bottom card, p_n is the top card.

In each step you pick some integer $k > 0$, take the top k cards from the original deck and place them, in the order they are now, on top of the new deck. You perform this operation until the original deck is empty. (Refer to the notes section for the better understanding.)

Let's define an *order of a deck* as $\sum_{i=1}^n n^{n-i} \cdot p_i$.

Given the original deck, output the deck with maximum possible order you can make using the operation above.

Input

The first line contains a single integer t ($1 \leq t \leq 1000$) — the number of test cases.

The first line of each test case contains the single integer n ($1 \leq n \leq 10^5$) — the size of deck you have.

The second line contains n integers $p_1, p_2, \dots, p_n$ ($1 \leq p_i \leq n; p_i \neq p_j$ if $i \neq j$) — values of card in the deck from bottom to top.

It's guaranteed that the sum of n over all test cases doesn't exceed 10^5 .

Output

For each test case print the deck with maximum possible order. Print values of cards in the deck from bottom to top.

If there are multiple answers, print any of them.

Examples

Input	Output
4 4 1 2 3 4 5 1 5 2 4 3 6 4 2 5 3 6 1 1 1	4 3 2 1 5 2 4 3 1 6 1 5 3 4 2 1

Note

In the first test case, one of the optimal strategies is the next one:

1. take 1 card from the top of p and move it to p' : p becomes $[1, 2, 3]$, p' becomes $[4]$;
2. take 1 card from the top of p : p becomes $[1, 2]$, p' becomes $[4, 3]$;
3. take 1 card from the top of p : p becomes $[1]$, p' becomes $[4, 3, 2]$;
4. take 1 card from the top of p : p becomes empty, p' becomes $[4, 3, 2, 1]$.

In result, p' has order equal to $4^3 \cdot 4 + 4^2 \cdot 3 + 4^1 \cdot 2 + 4^0 \cdot 1 = 256 + 48 + 8 + 1 = 313$.

In the second test case, one of the optimal strategies is:

1. take 4 cards from the top of p and move it to p' : p becomes $[1]$, p' becomes $[5, 2, 4, 3]$;
2. take 1 card from the top of p and move it to p' : p becomes empty, p' becomes $[5, 2, 4, 3, 1]$;

In result, p' has order equal to $5^4 \cdot 5 + 5^3 \cdot 2 + 5^2 \cdot 4 + 5^1 \cdot 3 + 5^0 \cdot 1 = 3125 + 250 + 100 + 15 + 1 = 3491$.

In the third test case, one of the optimal strategies is:

1. take 2 cards from the top of p and move it to p' : p becomes $[4, 2, 5, 3]$, p' becomes $[6, 1]$;
2. take 2 cards from the top of p and move it to p' : p becomes $[4, 2]$, p' becomes $[6, 1, 5, 3]$;
3. take 2 cards from the top of p and move it to p' : p becomes empty, p' becomes $[6, 1, 5, 3, 4, 2]$.

In result, p' has order equal to $6^5 \cdot 6 + 6^4 \cdot 1 + 6^3 \cdot 5 + 6^2 \cdot 3 + 6^1 \cdot 4 + 6^0 \cdot 2 = 46656 + 1296 + 1080 + 108 + 24 + 2 = 49166$.

Problem B. B

Time Limit 2000 ms

Mem Limit 262144 kB

AksLolCoding is playing a game on a string s of length n . Initially, s contains only lowercase Latin characters.

In one turn, AksLolCoding can choose any pair of integers (i, j) such that:

- $1 \leq i < j \leq n$;
- $s_i = s_j \neq *$; and
- $s_k = *$ for all $i < k < j$.

If no such i, j exists, then the game ends. Otherwise, AksLolCoding sets $s_i := *$ and $s_j := *$.

Once the game ends, AksLolCoding wins if and only if every character in s is equal to $*$. Determine if it is possible for AksLolCoding to win.

Note: $*$ represents ASCII character 42.

Input

The first line contains an integer t ($1 \leq t \leq 100$), the number of test cases.

The first line of each test case contains an integer n ($1 \leq n \leq 5000$), the length of the string.

The second line of each test case contains a string s consisting of lowercase Latin characters.

The sum of n over all test cases does not exceed 5000.

Output

Output the answer to each test case on its own line. If AksLolCoding can win, output "YES" (without quotes). Else, output "NO" (without quotes).

You can output the answer in any case (upper or lower). For example, the strings "yEs", "yes", "Yes", and "YES" will be recognized as positive responses.

Examples

Input	Output
6	NO
1	YES
a	YES
6	NO
llmllm	NO
6	YES
uwuuwu	
6	
byebye	
6	
oooioi	
12	
siixxsevvnn	

Note

In the first test case, it can be shown that it is impossible for AksLolCoding to win.

In the second test case, AksLolCoding can win by making the following moves: $llmllm \rightarrow llm**m \rightarrow **m**m \rightarrow ****$

In the third test case, AksLolCoding can win by making the following moves: $uwuuwu \rightarrow uw**wu \rightarrow u****u \rightarrow ****$

Problem C. C

Time Limit 1000 ms

Mem Limit 262144 kB

Input File stdin

Output File stdout

The classic programming language of Bitland is Bit++. This language is so peculiar and complicated.

The language is that peculiar as it has exactly one variable, called X . Also, there are two operations:

- Operation ++ increases the value of variable X by 1.
- Operation -- decreases the value of variable X by 1.

A statement in language Bit++ is a sequence, consisting of exactly one operation and one variable X . The statement is written without spaces, that is, it can only contain characters "+", "-", "X". Executing a statement means applying the operation it contains.

A programme in Bit++ is a sequence of statements, each of them needs to be executed. Executing a programme means executing all the statements it contains.

You're given a programme in language Bit++. The initial value of X is 0. Execute the programme and find its final value (the value of the variable when this programme is executed).

Input

The first line contains a single integer n ($1 \leq n \leq 150$) — the number of statements in the programme.

Next n lines contain a statement each. Each statement contains exactly one operation (++ or --) and exactly one variable X (denoted as letter «X»). Thus, there are no empty statements. The operation and the variable can be written in any order.

Output

Print a single integer — the final value of X .

Examples

Input	Output
1 ++X	1

Input	Output
2 X++ --X	0

Problem D. D

Time Limit 1000 ms

Mem Limit 262144 kB

Input File stdin

Output File stdout

Your friend has recently learned about coprime numbers. A pair of numbers $\{a, b\}$ is called *coprime* if the maximum number that divides both a and b is equal to one.

Your friend often comes up with different statements. He has recently supposed that if the pair (a, b) is coprime and the pair (b, c) is coprime, then the pair (a, c) is coprime.

You want to find a counterexample for your friend's statement. Therefore, your task is to find three distinct numbers (a, b, c) , for which the statement is false, and the numbers meet the condition $l \leq a < b < c \leq r$.

More specifically, you need to find three numbers (a, b, c) , such that $l \leq a < b < c \leq r$, pairs (a, b) and (b, c) are coprime, and pair (a, c) is not coprime.

Input

The single line contains two positive space-separated integers l, r ($1 \leq l \leq r \leq 10^{18}$; $r - l \leq 50$).

Output

Print three positive space-separated integers a, b, c — three distinct numbers (a, b, c) that form the counterexample. If there are several solutions, you are allowed to print any of them. The numbers must be printed in ascending order.

If the counterexample does not exist, print the single number -1.

Examples

Input	Output
2 4	2 3 4

Input	Output
10 11	-1

Input	Output
9000000000000000009 9000000000000000029	9000000000000000009 9000000000000000010 9000000000000000021

Note

In the first sample pair (2, 4) is not coprime and pairs (2, 3) and (3, 4) are.

In the second sample you cannot form a group of three distinct integers, so the answer is -1.

In the third sample it is easy to see that numbers 9000000000000000009 and 9000000000000000021 are divisible by three.

Problem E. E

Time Limit 1000 ms

Mem Limit 524288 kB

You are given an array a consisting of n positive integers. Find a **non-empty** subset of its elements such that their sum is **even** (i.e. divisible by 2) or determine that there is no such subset.

Both the given array and required subset may contain equal values.

Input

The first line contains a single integer t ($1 \leq t \leq 100$), number of test cases to solve. Descriptions of t test cases follow.

A description of each test case consists of two lines. The first line contains a single integer n ($1 \leq n \leq 100$), length of array a .

The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 100$), elements of a . The given array a can contain equal values (duplicates).

Output

For each test case output -1 if there is no such subset of elements. Otherwise output positive integer k , number of elements in the required subset. Then output k distinct integers ($1 \leq p_i \leq n$), indexes of the chosen elements. If there are multiple solutions output any of them.

Examples

Input	Output
3 3 1 4 3 1 15 2 3 5	1 2 -1 2 1 2

Note

There are three test cases in the example.

In the first test case, you can choose the subset consisting of only the second element. Its sum is 4 and it is even.

In the second test case, there is only one non-empty subset of elements consisting of the first element, however sum in it is odd, so there is no solution.

In the third test case, the subset consisting of all array's elements has even sum.

Problem F. F

Time Limit 2000 ms

Mem Limit 262144 kB

Input File stdin

Output File stdout

Vasya works as a DJ in the best Berland nightclub, and he often uses dubstep music in his performance. Recently, he has decided to take a couple of old songs and make dubstep remixes from them.

Let's assume that a song consists of some number of words. To make the dubstep remix of this song, Vasya inserts a certain number of words "WUB" before the first word of the song (the number may be zero), after the last word (the number may be zero), and between words (at least one between any pair of neighbouring words), and then the boy glues together all the words, including "WUB", in one string and plays the song at the club.

For example, a song with words "I AM X" can transform into a dubstep remix as "WUBWUBIWUBAMWUBWUBX" and cannot transform into "WUBWUBIAMWUBX".

Recently, Petya has heard Vasya's new dubstep track, but since he isn't into modern music, he decided to find out what was the initial song that Vasya remixed. Help Petya restore the original song.

Input

The input consists of a single non-empty string, consisting only of uppercase English letters, the string's length doesn't exceed 200 characters. It is guaranteed that before Vasya remixed the song, no word contained substring "WUB" in it; Vasya didn't change the word order. It is also guaranteed that initially the song had at least one word.

Output

Print the words of the initial song that Vasya used to make a dubstep remix. Separate the words with a space.

Examples

Input	Output
WUBWUBABCWUB	ABC

Input	Output
WUBWEWUBAREWUBWUBTHEWUBCHAMPIONSWUBMYWUBFRIENDWUB	WE ARE THE CHAMPIONS MY FRIEND

Note

In the first sample: "WUBWUBABCWUB" = "WUB" + "WUB" + "ABC" + "WUB". That means that the song originally consisted of a single word "ABC", and all words "WUB" were added by Vasya.

In the second sample Vasya added a single word "WUB" between all neighbouring words, in the beginning and in the end, except for words "ARE" and "THE" — between them Vasya added two "WUB".

Problem G. G

Time Limit 2000 ms

Mem Limit 262144 kB

You are given an array a of size n and an integer k . You do the following procedure k times:

- For each element a_i , you set a_i to $\text{mex}^*(a_1, a_2, \dots, a_{i-1}, a_{i+1}, a_{i+2}, \dots, a_n)$. In other words, you set a_i to the mex of all other elements in the array. **This is done for all elements in the array at the same time.**

Please find the sum of elements in the array after all k operations.

*The minimum excluded (MEX) of a collection of integers $d_1, d_2, \dots, d_k$ is defined as the smallest non-negative integer x which does not occur in the collection d .

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). The description of the test cases follows.

The first line contains two integers n and k ($2 \leq n \leq 2 \cdot 10^5, 1 \leq k \leq 10^9$) – the number of elements in a and the number of operations done.

The second line contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq n$).

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output the sum of elements after all k operations on a new line.

Examples

Input	Output
5 3 3 0 2 1 2 4 0 2 4 1 0 0 1 1 8 7 6 6 2 4 3 0 1 8 2 2 0 0	3 1 8 25 0

Note

In the first test case, we performed the operation on the array $[0, 2, 1]$ three times. Let's compute the result after the first time:

- The first element becomes $\text{MEX}(2, 1) = 0$
- The second element becomes $\text{MEX}(0, 1) = 2$
- The third element becomes $\text{MEX}(0, 2) = 1$

So, after the first operation, $[0, 2, 1]$ becomes $[0, 2, 1]$ again. It can be shown that the array will not change no matter how many times we perform the operation, so the final array after three operations is still $[0, 2, 1]$. The sum is $0 + 2 + 1 = 3$.

In the third test case, the array becomes $[2, 2, 2, 2]$.